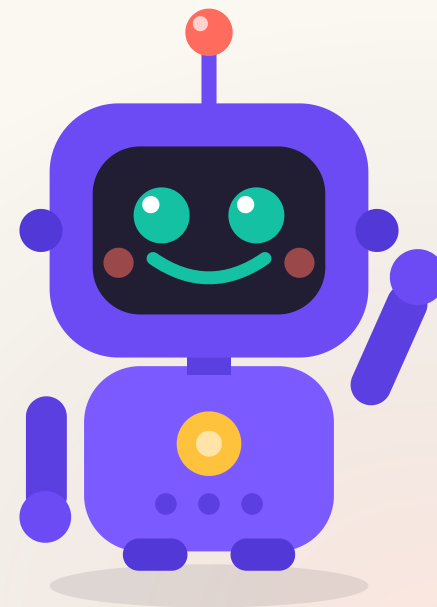


МОДУЛЬ 6 · УРОК 6.2

Собираем проект по шагам

Превращаем план в
работающую
программу и учимся
ловить ошибки.



Пора оживить план



У тебя уже есть план — теперь оживим его! Будем класть кирпичик за кирпичиком. А если что-то сломается — не страшно, чинить ошибки я тебя научу.

Сегодня план превращается в настоящую программу.

Программа не работает? Это нормально.

Каждый программист чинит ошибки. Сегодня научимся это делать спокойно.



Вспомним прошлый урок

- Выбрали идею проекта и записали её в одно предложение
- Разбили задачу на **4–6 маленьких шагов**
- Поняли правило: один шаг → запустил → работает → дальше

✨ А ЗНАЕШЬ ЧТО?

Даже большие команды программистов работают точно так же: маленькими шагами и с проверкой после каждого. Ты делаешь по-настоящему!



План урока

- 1 Как собирать проект по шагам
- 2 Где искать ошибку, когда «не работает»
- 3 Два вида ошибок: компилятор и логика
- 4 Частые проблемы и их решения
- 5 **Практика в классе** — работаем над своим проектом
- 6 **Домашнее задание**

Собираем по одному шагу

Берём первый шаг плана, пишем, запускаем — и только потом следующий.

main.cpp

```
// Шаг 1: поздороваться  
std::cout << "Игра-викторина! Готов?\n";  
// запустили — работает? отлично, пишем шаг 2
```

💡 СОВЕТ

Не пиши сразу всё. Каждый рабочий шаг — это **+1 к уверенности**. Маленькие победы складываются в большой проект.

Этапы сборки проекта



Пишу шаг

один кусочек



Запускаю

проверяю



Сохраняю

чекпоинт

Прошёл круг — берёшься за следующий шаг. И так до финала.

Чекпоинты — точки сохранения

Программа работает после шага? Это чекпоинт.



Как в игре: дошёл до чекпоинта — закрепился. Если дальше что-то сломаешь, всегда можно вернуться к последнему рабочему месту. Сохраняй файл после каждого работающего шага!



Два вида ошибок

Ошибка компилятора

программа **не запускается**: забыл `;`,
кавычку, скобку

Ошибка логики

программа запускается, но **считает
неправильно**

 СОВЕТ

Первую покажет g++ красным текстом — он сам подскажет строку. Вторую ищем сами: глазами и проверкой.

Как читать ошибку компилятора

```
main.cpp  
  
std::cout << "Привет"    // забыли ;  
std::cout << "Пока";
```

g++ напишет что-то вроде: `error: expected ';' before 'std'`

СОВЕТ

Главное — **номер строки** в ошибке. Смотри на эту строку и строку **над** ней: часто пропущенный знак прячется именно там.

Где искать ошибку логики

```
main.cpp

int score = 0;
if (answer == "Париж") score = 1;    // а не score + 1!
std::cout << "Очков: " << score;    // всегда 1?
```

Программа работает, но считает не так. Проверь её «руками».

✨ А ЗНАЕШЬ ЧТО?

Ответил правильно дважды, а очков всё равно 1? Значит, нашёл ошибку логики. Программа честно делает то, что написано — просто написано не то.

Метод «печатаем подсказки»

Не знаешь, что внутри переменной? Выведи её!

main.cpp

```
std::cout << "[отладка] score = " << score << "\n";
```

 ПОПРОБУЙ

Вставь временный `std::cout` прямо сейчас в свой проект и посмотри, какое значение на самом деле. Нашёл ошибку — убрал подсказку.

Частые проблемы и решения ⚠



`error: expected`

`';'`

Забыл точку с запятой. Добавь `;` в конце строки.



Окно сразу закрылось

Так и должно быть — это не ошибка. Запускай из терминала.



`cin` не читает строку

Имя из двух слов? Используй `std::getline`.



Считает неправильно

Проверь `=` против `==` и `score = score + 1`.

`getline` для длинного ответа

`std::cin >>` читает одно слово. Для целой фразы — `std::getline`.

```
main.cpp

std::string name;
std::cout << "Как тебя зовут? ";
std::getline(std::cin, name);
std::cout << "Привет, " << name << "!";
```

Удобно, когда ответ из нескольких слов: «Нижний Новгород».



Советы от опытных

- Запускай **часто** — после каждого маленького шага
- Имена переменных — понятные: `score`, а не `s`
- Сломалось? Вспомни, **что менял** последним
- Не получается час — спроси или отложи, отдохни
- Сохраняй рабочую версию перед большими правками

Чек-лист: «застрял — что делать»



Читай ошибку

Текст ошибки и номер строки — твоя карта.



Смотри выше

Загляни и в строку **над** ошибкой.



Печатай подсказки

Вставь `std::cout` и проверь значения.



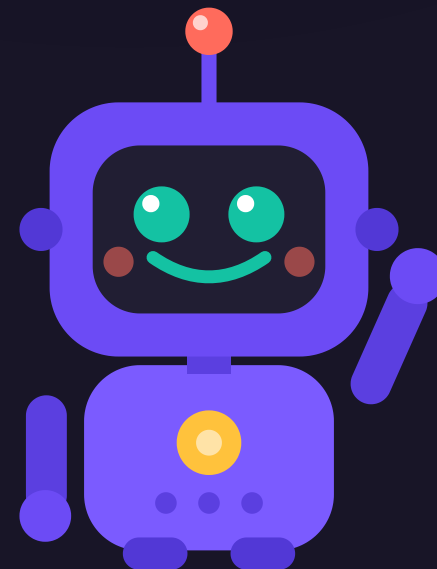
Вернись к чекпоинту

Отката к последней рабочей версии.

ПРАКТИКА В КЛАССЕ

Работаем над проектом

Берём свой план и собираем программу шаг за шагом.



✦ ЗАДАЧА

Задача 1 · Первый чекпоинт

Собери **первые два шага** своего проекта в одну программу.

- Шаг за шагом: написал → запустил → работает
- Дошёл до рабочей версии — сохрани файл

Время: 10 минут

✦ ЗАДАЧА

Задача 2 · Почини код

В этом коде **три ошибки**. Найди и исправь:



main.cpp

```
int main() {  
    int score = 0  
    std::cout << "Вопрос: 2 + 2?";  
    std::cin >> answer;
```

Подсказка: одна про `;`, одна про объявление переменной

✦ ЗАДАЧА

Задача 3 · Добавь ещё шаг +

Допиши к проекту **следующий шаг** из своего плана.

- Например: счётчик очков или ещё один вопрос
- После каждого добавления — запусти и проверь

Время: 10 минут

✦ ЗАДАЧА

Задача 4 · Отладка подсказками

★ со звёздочкой

Найди в своём проекте место, где «что-то не так», и вставь временный `std::cout` с подсказкой.

- 1 Выведи значение переменной
- 2 Найди ошибку и убери подсказку

Кто поймает хитрую ошибку – расскажет классу, как нашёл!

Что мы сегодня научились 🦾



Теперь ты не боишься ошибок — ты умеешь их ловить и чинить. Это и есть настоящее программирование!

- Собирать проект по шагам и делать чекпоинты
- Различать ошибки компилятора и ошибки логики
- Читать текст ошибки и смотреть на номер строки
- Искать ошибки методом «печатаем подсказки»

Домашнее задание

1. Доделай проект по плану — собирай его по шагам.
2. Заполни рабочий лист 6.2 и держи рядом шпаргалку 6.2.

Дальше: урок 6.3 — готовим защиту и показываем проект классу.